Collision-free Encoding for Chance-constrained, Non-convex Path Planning

Marcio da Silva Arantes, Claudio F. M. Toledo, Brian Charles Williams, Masahiro Ono

***Abstract*—The path planning methods based on non-convex constrained optimization, such as mixed-integer linear programming (MILP), have found various important applications, ranging from unmanned aerial vehicles (UAVs) [1] and autonomous underwater vehicles (AUVs) [2] to space vehicles [3]. Moreover, their stochastic extensions have enabled risk-aware path planning, which explicitly limits the probability of failure to a user-specified bound. However, a major challenge of those path planning methods is constraint violation between discrete time steps. In the existing approach, a path is represented by a sequence of waypoints and the safety constraints (e.g., obstacle avoidance) are imposed on waypoints. Therefore, the trajectory between waypoints could violate the safety constraints. A naive continuous-time extension results in unrealistic computation cost. We propose a novel approach to ensure constraint satisfaction between waypoints without employing a continuous-time formulation. The key idea is to enforce that the same inequality constraint is satisfied on any two adjacent time steps, under assumptions of polygonal obstacles and straight line trajectory between waypoints. The resulting problem encoding is MILP, which can be solved efficiently by commercial solvers. Thus, we also introduce novel extensions to risk-allocation path planners with improved scalability for real-world scenarios and runtime performance. While the proposed encoding approach is general, the particular emphasis of this paper is placed on the chance-constrained, non-convex path-planning problem (CNPP). We provide extensive simulation results on CNPP to demonstrate the path safety and scalability of our encoding and related path planners.**

***Index Terms*—Autonomous agents, chance constraints, optimization under uncertainty, probabilistic planning, mixed-integer linear programming.**

I. Introduction

Real-world path planning problems often have complex, non-convex state constraints for the sake of safety. Take, for example, the airspace around Tokyo's two airports shown in Fig. 1, which constitute the third busiest city airport system in the world. Commercial airplanes take off from and land on the airports must keep off two airspaces reserved for nearby military airbases, shown in green and blue in the figure. The airspaces have a non-convex, three-dimensional shape, where the ceiling height varies discontinuously like stairs. Various other safety constraints must also be respected, such as spacial separation between airplanes and the scheduling constraints of runways. Failure to satisfy these constraints could result in critical consequences.

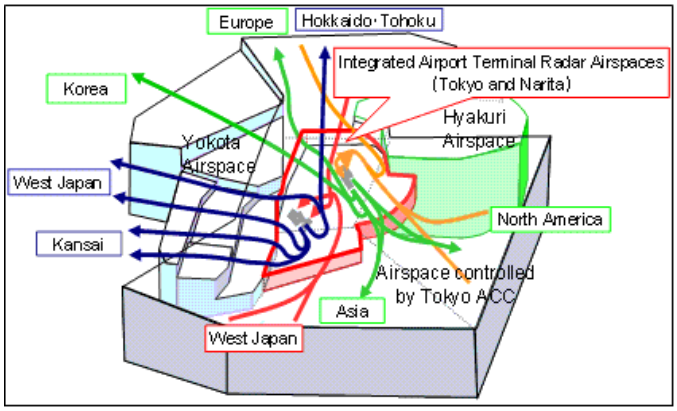


Fig. 1. Airspace around Tokyo, Japan. 3-D constraints. Source: Japanese Ministry of Land, Infrastructure, Transport and Tourism website.

An efficient path planning approach for dynamic systems to handle nonconvex state constraints is to formulate the problem as model predictive control (MPC) [4]. It plans discrete-time control and state sequences concurrently through constrained optimization. The optimization problem to solve is mixed integer linear programming (MILP) when the dynamics is linear, obstacles are represented by combinations of polytopes, and no uncertainty is present. If the system is subject to stochastic uncertainty, a viable approach is path planning with chance-constraints [5], which cap the probability of constraint violations.

An outstanding problem of this approach is "jumps" between time steps. Since the approach only concerns the constraint satisfaction at discrete points of time, the resulting path could cut through obstacles as shown in Fig. 2-Left. In fact, such undesirable "jumps" are commonly observed both in deterministic and stochastic settings because "jumps" often help to reduce typical cost functions, such as the path length and the control effort. This issue could be mitigated, but not fully addressed, by increasing sampling frequency (i.e., reduced time intervals between time steps) at a cost of increased computation time. Another existing approach is to set safety margin around the obstacles [6], but it introduces conservatism. More importantly, this approach works only when the distance between neighbor states is upper-bounded.

Marcio da Silva Arantes are with SENAI, Innovation Institute of Embedded Systems (e-mail: marcio.arantes@sc.senai.br).

Claudio F. M. Toledo are with University of Sao Paulo, Institute of Mathematics and Computer Science (e-mail: claudio@icmc.usp.br).

Brian Charle Williams is with Massachusetts Institute of Technology, Computer Science and Artificial Intelligence Laboratory (e-mail: williams@csail.mit.edu).

Masahiro Ono is with California Institute of Technology, Jet Propulsion Laboratory (e-mail: masahiro.ono@jpl.nasa.gov).

Manuscript received YYY XX, 20XX; revised YYY XX, 20XX.

This research was supported by Fundacao de Amparo a Pesquisa do Estado de Sao Paulo (FAPESP), projects 2014/12297-0, 2014/11331-0 and 2013/07375-0, and by the Office of Naval Research Grant N00014-15-IP-00052. The research described in this paper was carried out in part at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration.

The contribution of this paper is two-fold. First, we propose a simple yet powerful approach to fully suppress the "jumps" without incurring significantly increased computation time or conservatism. The main idea is to impose constraints that require every pair of adjacent states to be on the same "side" of an obstacle, as shown in Fig. 2-Right. The original set of obstacle avoidance constraints are replaced by the new set of constraints as the latter is an sufficient condition of the former.

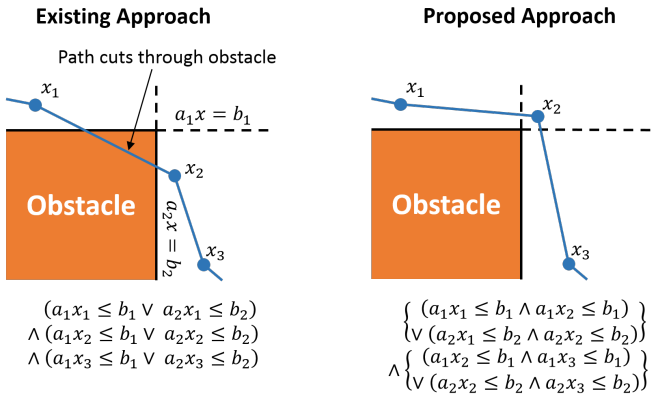


Fig. 2. Left: Conventional MPC-based path planning requires that states are outside of polygonal obstacles at each time step. As a result, path between time steps could cut through obstacles. Right: Instead, the proposed approach requires that each state pair at adjacent time steps are on the same side of the obstacles. The resulting path is guaranteed to be outside of the obstacles even between time steps. In the figure, the two sides of the obstacle are part of hyperplanes represented by $a_ix = b_i$. A state x is outside of the obstacle if $a_1x \le b_1 \vee a_2x \le b_2$.

The second contribution of this paper is to extend this approach to chance-constrained, non-convex path planning (CNPP) improving scalability and run-time performance for real-world scenarios. We specifically build upon the risk-allocation approach, where the allocation of risk among time steps and constraints are optimized through iterative optimization in order to minimize the conservatism in the handling of chance constraints [7]. From the linear encoding proposed, extensions to risk-allocation path planners are improved as well as a novel risk-allocation path planner is introduced.

The remainder of this paper is organized as follows: Section 2 reviews the main pieces of prior work that are related to this study. We formally define CNPP in Section 3, and then present the proposed constraint encoding in Section 4. The solution methods are described in Section 5 and the simulation results are reported in Section 6. Finally, the conclusions of this work follows in Section 7.

II. Related Work

There are several techniques in the literature for path planning problems [8]–[10]. In the Potential Field Methods (PFMs), the autonomous vehicle behaves as a particle attracted by goals and repulsed by obstacles. The PFMs are computationally efficient and mathematically simple to implement, while their disadvantages are to drop in local minima, to fail for scenarios with obstacles close to each other and oscillations in narrow passage [8]. The Sampling-Based motion Planning (SBP) methods, such as Probabilistic Roadmap Method (PRM) [11] and Rapidly-exploring Random Tree (RRT) [12], are capable of generating feasible trajectory under hard avoidance constraints. SBP methods can deal better with time critical and complex planning problems. The main disadvantage is to return suboptimal solutions [10].

The new encoding introduced in our paper aims a better description of the problem than those achieved using SBPs and PFM. Therefore, MILP optimization-based frameworks are proposed to find optimal or near optimal solutions. The new encoding has limitation for real time (online) applications when compared against SBPs and PFM. The authors in [13] report the lack of scalability for MILP optimization-based frameworks, since they are NP-hard in the number of binary variables. In this context, they argue favorably to apply SBP methods. However, one contribution of our paper is to evaluate how scalability can be enhanced by our encoding.

The problem approached by the present paper is the CNPP introduced by [14] for UAV. CNPP addresses path planning with uncertainty within a full observable and known geometric environment. In this type of environment, the authors in [11] maximize the chance to reach a goal without collisions in the presence of uncertainty related to robot motion. Stochastic Motion Roadmap (SMR) is combined with Markov Decision Process (MDP), where the roadmap with collision-free states is first built. MDP is defined from this roadmap and solved in polynomial time applying dynamic programming.

The uncertainty in ocean current is approached by [15] when planning risk-aware paths for Autonomous Underwater Vehicles (AUVs). Two probabilistic planners are proposed, where the confidence is measured using stationary and non-stationary Gaussian process model based on the MDP. Results are reported from extensive simulations as well as field tests using Slocum gliders. The stationary model is able to improve safety and reliability in slower ocean currents, while the non-stationary model is better for faster ocean currents. The authors point out the need to make the method scalable for larger environments. This is exactly the aspect evaluated by the solution methods reported in our paper: scalability for large environments. However, we propose a MILP approach instead of models based on MDP.

The authors in [16] combine Rapidly-Exploring Random Tree (RRT) and Linear-Quadratic controller with Gaussian models for Motion Planning (LQG-MP) under uncertainty. LQG-MP defines the a-priori probability distributions for states, which allow evaluating the path for obstacle avoidance, while RRT gives the possible paths. A similar approach to deal with uncertainty is described in [17]. The LQG controller is improved by Rapidly-exploring Random Belief Tree (RRBT) method.

A Partially Observable Markov Decision Processes (POMDPs) is studied in [18] and [19] with Stochastic Differential Dynamic Programming (SDDP) applied by [18], while an iterative LQG (iLQG) is used in [19]. The Stochastic Extended Linear Quadratic Regulator (SELQR) for belief states is described in [20], where a linear description for the stochastic dynamics and a quadratic description for local cost functions are employed.

The methods proposed in [18]–[20] can be applied to

problems where nonlinear dynamics happen and action and state spaces are continuous. In these problems, the belief state can be represented using a Gaussian distribution within an environment that can include obstacle. Models related to dynamics and observations as well as cost functions must be smooth. However, the maximum-likelihood estimation is not mandatory when computing control policies and fixed estimator or control gains are not assumed.

There are some differences between our MILP encoding and the LQG methods mentioned. The obstacle avoidance, control applied and level of uncertainty are imposed by linear constraints in our encoding and there is no penalties related to deviation from desired states. The value function of iLQG may demand parameters for obstacle avoidance based on human knowledge as explained by [21], while our encoding relies only on the position of obstacle. To sum up, the proposed encoding introduces a "hard" probabilistic guarantee on constraint violation, while the alternative models of path planning under uncertainty (MDP, iLQG, etc) provides "soft" trade-offs (through the reward function), but not hard guarantee. However, one limitation of the chance-constrained approach is computational scalability, which this paper tries to address.

The authors in [22] argue that mathematical model based frameworks can describe the path planning problem as a time variant system, where obstacles and kinodynamic constraints are simultaneously taken into account. The thesis in [23] introduces concepts and algorithms for online receding horizon trajectory. The work in [24] applies the concepts from [23] and a three dimension formulation is introduced. Constraints and variables are defined at each time step to reduce complexity by solving MILPs. Horizon minimization and variable time step improve computational performance, while linear interpolation are used to avoid conservative trajectories. No-fly-zones are described as obstacles in [6], where the main idea is to expand the obstacle sizes. This expansion will define a safety distance when building a trajectory. A convex model for path planning is formulated in [25], where the autonomous vehicle must keep a safety distance from boundaries through the convex region.

The works in [6], [23]–[26] do not deal with uncertainty, which is done explicitly by [14] for the CNPP. The risk allocation is applied in [14] to solve the path planning problem within non-convex region. The problem is first encoded as a stochastic non-convex and non-linear mathematical programming model where chance constraints are proposed. Next, the Customized Solution Approach (CSA) is designed from CNPP encoding. The CSA combines upper and lower bounds solutions for MILP and LP formulations derivated from the non-convex and non-linear mathematical programming model. In [3] and [5], solution methods are proposed for path planning problems in real-world environments using UAVs, autonomous underwater vehicles and spacecraft. These studies are based on the initial ideas introduced in [14].

The CNPP encoding in [14] may exceed in practice the allocated risk. For example, using the formulation described in [3], [5], [14], the vehicle position at two successive time steps is feasible, but the trajectory between them can intersect an obstacle as previously illustrated in Figure 2. The solution methods in [5], [14] also become computationally prohibitive when the environment includes a large number of obstacles and the planning convers a long time horizon. This problem is left as a future work by the authors in [5]. Moreover, one of the contributions of the current paper is to report the improvement achieved in terms of scalability applying the new CNPP encoding.

The recent works [21], [27] ensure safety trajectories using mathematical model based approach. The authors in [27] generate safe regions by overlapping convex areas such that each trajectory segment is on the same convex region. The collision free path is planned for a quadrotor in [21], where the vehicle must be on the same side of an obstacle to avoid collision. However, there are relevant differences between the current paper and [21], [27]. First, we approach collision free path with uncertainty. Second, different from [27], the proposed CNPP encode describes obstacles directly without convex transformation. The transformation applied in [27] creates a small number of convex areas by losing part of the feasible region. A greedy tool named IRIS is applied for computational tractability. The authors mention that dealing directly with obstacles can become intractable with more than a handful of simple obstacles. The results reported in our paper are obtained from a large amount of obstacles without compromise tractability for the majority of scenarios. Third, the quadrotor in [21] are on the same face of a obstacle, which does not happen for consecutive time steps as proposed by us.

III. Problem Statement

Let's suppose we have an UAV that starts from one airport and reaches another airport located behind some mountains as shown by Figure 3. It is not desired that such aircraft fly over populated areas, regular airports or storm regions, these are specified as no-fly zones (NFZ). The aircraft can fly, for example, over forests and mountains, but uncertainties related to the environment and the UAVs dynamics can lead to disturbances during the flight. A path planner that operates

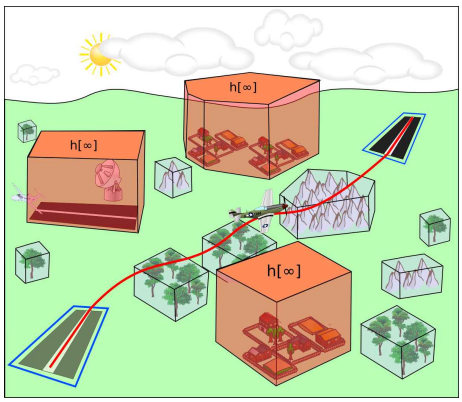


Fig. 3. Path planning through non-convex regions.

under this situation must contend with a certain level of risk that the UAV goes through a NFZ or collides with a mountain. The task of the planner is to find a path for the UAV that terminates on the destination, while optimizing a measure such as fuel consumption or distance, without violating a maximum level of allowed risk. In our treatment here, NFZs are viewed

as obstacles with unlimited height (h[∞]), and avoidance of these regions is treated as a hard constraint. Mountains and forests are also considered obstacles, but with limited height, as illustrated by Figure 3, hence it is possible for the UAV to fly over these obstacles. Given this treatment, path planning is performed over a non-convex region, represented by obstacles, along with a maximum acceptable risk of going through such obstacles. We call this a Chance-constraint Non-convex Path-planning Problem (CNPP) as first described in [14].

The inputs of the CNPP include initial position ($\hat{x}_0$), goal position ($\hat{x}_{goal}$), a stochastic plant model (SPM), time horizon (T), time interval (Δt), external uncertainties w_t and level of risk (Δ). The decision variables are system states (x_t) and controls (u_t) applied at each time step (t) to change these states in the plant (physical system) following the SPM. Thus, x_t defines the state of the plant at time step t, while u_t is the action or control applied over x_t that changes the current state. The probabilistic state transitions in the SPM are approximated by a discrete and linear recursive equation over x_t and u_t, where external uncertainties are added by w_t. A convex objective function g($\cdot$) is assumed over states x_t and controls u_t. The output of the CNPP is the optimal sequence of controls $S = (u_0^*, ..., u_T^*)$ applied over the system that is able to reach a goal position, without violating the level of risk, minimizing some measure defined by g($\cdot$).

The primary limitation of our approach for CNPP is the assumptions of polygonal obstacles and straight-line paths between waypoints, which are standard assumptions employed by most MPC-based path planning methods. These assumptions are justified by the fact that, in many practical applications, if not all, vehicle paths and keep-out/keep-in zones consist of simple geometries for operational convenience. For example, typical no-fly zones in airspace, such as the ones in Fig 1, are polygonal. Keep-in zones for Mars rovers' on-board path planning are also polygonal, as shown in Fig. 4-Left. In various applications, ranging from aircraft to Mars rovers, paths are often composed of straight lines and sharp turns, or turns in place in case of Mars rovers (Fig. 4-Right). If non-convex polygonal obstacles occur, they could be decomposed into several convex polygons as proposed by our encoding. The contribution of this paper has a practical importance because it provides a safety guarantee along a continuous path, with the comparable computational complexity as the discrete-time path optimization, in both deterministic and stochastic settings.

IV. Problem Encoding Improvements

This section introduces the improvements proposed to encode the CNPP defined on section III, which are described over the previous encoding introduced in [14].

A. Control plant and chance-constraints

First, the time horizon T is divided into time steps $t = 0, ..., T$, using a fixed time interval Δt between consecutive times steps. These time steps guide the vehicle from its initial state to its final state (goal position), while avoiding obstacles $j = 1, ..., J$. We assume that the initial position follows a

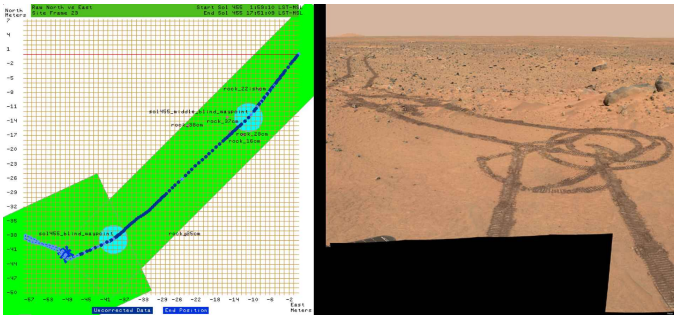


Fig. 4. Left: Polygonal keep-in zones for the autonomous path planning of Mars Rover Curiosity on Sol 455. Right: While Mars rovers have capabilities to drive on constant-curvature arcs, actual paths often consist of straight lines and turns in place, as shown in this figure.

Gaussian distribution with mean $\hat{x}_0$ and covariance matrix Σ_{x_0}, so $x_0 \sim \mathcal{N}(\hat{x}_0, \Sigma_{x_0})$. Moreover, uncertainties in the plant model are related to the initial position and external factors w_t, which act over time in the system dynamic with $w_t \sim \mathcal{N}(0, \Sigma_{w_t})$. Thus, the authors in [14] defined exactly the distribution of the future states following constraints (1) as stochastic plant model SPM. The mean μ_t of the current state x_t is linearly defined from control inputs $u_0, ..., u_{t-1}$, while the covariance at x_t (Σ_t) is not related to previous control inputs by equation (2). The covariance is determined from the initial state covariance (Σ_{x_0}) and noise covariances (Σ_{w_t}), which are previously known. The mean of the final state of the vehicle has to reach the goal state following constraint (3).

$$\mu_t = A^t \hat{x}_0 + \sum_{s=0}^{t-1} A^{t-s-1} B u_s \qquad \forall(t) \qquad (1)$$

$$\Sigma_t = A^t \Sigma_{x_0} (A^T)^t + \sum_{s=0}^{t-1} A^s \Sigma_{w_t} (A^T)^s \qquad \forall(t) \qquad (2)$$

$$\mu_T = \hat{x}_{goal} \qquad (3)$$

The chance-constraint in Equation (4) imposes a constraint on acceptable probability of flying over a NFZ and hitting an obstacle. The obstacles are defined by using disjunctive sets G_j of linear constraints, where $\bigwedge$ and $\bigvee$ are the logical *AND* and *OR* operators. Figure 5 depicts an obstacle avoidance, where the vehicle's position must be outside of at least one surface.

$$Pr(\bigwedge_{t=0}^{T} \bigwedge_{j=1}^{J} \bigvee_{i \in G_j} a_i^T x_t \geq b_i) \geq 1 - \Delta \qquad (4)$$

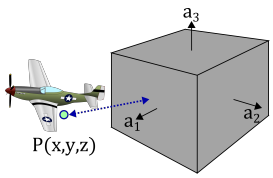


Fig. 5. Vehicle's position must be outside of at least one surface

This constraint guarantees that the level of risk incurred, when planning a path, will not violate the allowed risk bound

Δ in the CNPP. However, for a Gaussian probability distribution function $Pr(.)$, constraint (4) becomes non-linear. In general, the problem involves a multivariate Gaussian random variable x_t for the vehicle state at time step t, with variable mean μ_t and known covariance matrix Σ_t. In this case, the chance constraint can be reformulated as a deterministic constraint:

$$X \sim N(\mu, \sigma^2) \tag{5}$$

$$p(X < 0) \leq \delta \Leftrightarrow \mu \geq c. \tag{6}$$

where X is a single-variate Gaussian with mean μ, fixed variance σ^2 and c is the deterministic constraint. Thus, we can now rewrite equation (4) as equations (7)-(9): .

$$Pr(a_i^T x_t - b_i < 0) \leq \delta_{jt} \quad \Leftrightarrow \quad a_i^T \mu_t - b_i \geq c_{i,t}(\delta_{jt}) \tag{7}$$

$$c_{i,t}(\delta_{jt}) = \text{erf}^{-1}(1 - 2\delta_{jt})\sqrt{2a_i^T \Sigma_t a_i} \tag{8}$$

$$\text{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \tag{9}$$

Equation (7) reformulates the chance-constraint as a inequality over mean μ_t and a safety margin $c_{i,t}(\delta_{jt})$ that is a function of risk allocation δ_{jt}. At this point two aspects must be considered. First, to guarantee obstacle avoidance, the vehicle must be on the "outside" of at least one side of the obstacle ($\bigvee_{i \in G_j} a_i^T x_t \geq b_i$). Second, the safety margin is determined by a non-linear function, given by equations (8) and (9). The values of function $\text{erf}^{-1}(.)$ can be obtained using a look-up method, where we restrict such function to a convex region by keeping $\Delta \leq 0.5$. This means a probability of failure of less than 50% in equation (8) as proposed in [14].

The obstacle avoidance can be encoded by using binary variables (Z_{jti}) to specify the obstacle side for which the vehicle maintains a positive distance. This is achieved by constraints (10) and (11) where $\bar{M}$ is a positive big number. Constraints (10) engage the safety margin for the selected obstacle side ($Z_{jti} = 1$), while constraints (11) guarantee that this will happen for at least one side of each obstacle. Constraint (12) will satisfy the global chance constraint, where the sum of each risk δ_{jt} allocated to obstacle j at time step t can not exceed the allowed risk Δ. The derivation of Constraints (10)-(12) is described in [14].

$$c_{i,t}(\delta_{jt}) + b_i - a_i^T \mu_t \leq \bar{M}(1 - Z_{jti}) \qquad \forall (j, t, i \in G_j) \tag{10}$$

$$\sum_{i \in G_j} Z_{jti} \geq 1 \qquad \forall (j, t) \tag{11}$$

$$\sum_j \sum_t \delta_{jt} \leq \Delta \tag{12}$$

The authors in [14] did not deal with non-linearity when solving CNPP from their full encode; instead, relaxation approaches were proposed. Also, constraints (10) and (11) can lead to vehicle trajectories that exceed risk bounds specified in the problem formulation. The next sections describes improvements to solve such issues.

B. Improvements over chance constraints

As mentioned in section I an issue arises in the soundness of this formulation due to the discrete time formulation. It is possible to have a path, chosen, for example, to save fuel, that incurs a total risk greater than the maximum allowed risk Δ. Figure 6(a) shows another, more extreme situation, while Figure 6(b) illustrates a possible solution that restores the risk assessment to a conservative bound.

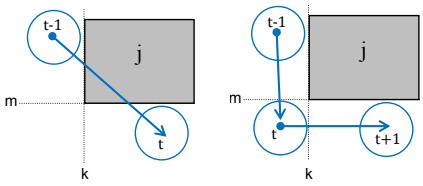


Fig. 6. (a) The vehicle could go trough the obstacle (b) Vehicle can not go trough it.

The previous encoding from constraints (10) and (11) in section IV-A under estimates risk, where a constraint is enabled at one time step and becomes disabled at the next time step. For example, as shown by Figure 6(a), suppose that at time step $t-1$, constraint related to side k is enabled and constraint m is disabled for obstacle j with $Z_{j,t-1,k} = 1$ and $Z_{j,t-1,m} = 0$. Furthermore, assume that at the next time step t, they switch values $Z_{j,t,k} = 0$ and $Z_{j,t,m} = 1$, . Constraints (10) and (11) are not violated, as presented by expressions (13), thus, no constraint prevents the vehicle from moving through the obstacle.

$$\left\{ \begin{array}{ccc} t-1 & : & \sum_{i \in G_j} Z_{j,t-1,i} = Z_{j,t-1,k} = 1 \geq 1 \\ & & c_{k,t-1}(\delta_{jt-1}) + b_{jk} - a_{jk}^T \mu_{t-1} \leq 0 \\ t & : & \sum_{i \in G_j} Z_{jti} = Z_{jtm} = 1 \geq 1 \\ & & c_{m,t}(\delta_{jt}) + b_{jm} - a_{jm}^T \mu_t \leq 0 \\ & & \textbf{feasible} \end{array} \right\} \tag{13}$$

To avoid such a situation, if a constraint k is enabled at time step $t-1$ ($Z_{j,t-1,k} = 1$), we introduce an additional condition that requires constraint k to continue to hold up through time t ($Z_{j,t,k} = 1$). This imposes the constraint that the vehicle must remain outside of constraint k between time steps $t-1$ and t, thus leading the vehicle along the corresponding edge of the obstacle (Figure 6(b)), instead of jumping from one side directly to another side (Figure 6(a)). With the added requirement that constraint k persist between the successive time points, the probability of failure estimate is guaranteed to be conservative across continuous time.

Lemma - For all obstacles j and time steps t, to encode obstacle avoidance, we need to ensure that $\exists i \in G_j$ such that $Z_{j,t,i} \wedge Z_{j,t-1,i} = 1$. This occurs, if and only if constraints (14)-(18) hold. Mathematically we have:

.

$$\bigvee_{i \in G_j} (Z_{j,t,i} \wedge Z_{j,t-1,i}) = 1 \quad \Leftrightarrow \quad \textit{(14)-(18) hold}$$

$$\sum_{i \in G_j} p_{j,t,i} \geq 1 \tag{14}$$

$$p_{j,t,i} \geq Z_{j,t,i} + Z_{j,t-1,i} - 1 \tag{15}$$

$$p_{j,t,i} \leq Z_{j,t,i} \tag{16}$$

$$p_{j,t,i} \leq Z_{j,t-1,i} \tag{17}$$

$$p_{j,t,i} \geq 0 \tag{18}$$

Proof. If $\exists i \in G_j$ such that $(Z_{j,t,i} \wedge Z_{j,t-1,i}) = 1$

$$\Leftrightarrow \sum_{i \in G_j} (Z_{j,t,i} \wedge Z_{j,t-1,i}) \geq 1 \Leftrightarrow \sum_{i \in G_j} p_{j,t,i} \geq 1$$

where $p_{j,t,i} = Z_{j,t,i} \wedge Z_{j,t-1,i}$. Thus, we can have:

$$p_{j,t,i} = Z_{j,t,i} \wedge Z_{j,t-1,i} \Leftrightarrow$$

$$\left\{ \begin{array}{rl} p_{j,t,i} \geq Z_{j,t,i} + Z_{j,t-1,i} - 1 & \forall i \in G_j, \forall j, (t > 0) \\ p_{j,t,i} \leq Z_{j,t,i} & \forall i \in G_j, \forall j, (t > 0) \\ p_{j,t,i} \leq Z_{j,t-1,i} & \forall i \in G_j, \forall j, (t > 0) \end{array} \right\}$$

□

Returning to our problematic example, shown in Figure 6(a), the new encoding (14)-(18) will be infeasible as shown by expression (19).

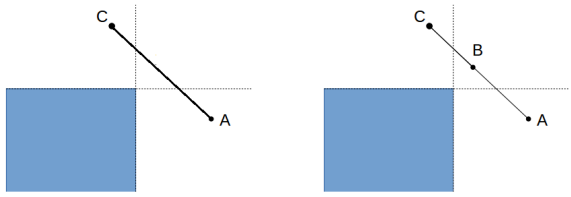
$$\left\{ \begin{array}{c} p_{j,t,k} = (Z_{j,t,k} \wedge Z_{j,t-1,k}) = (0 \wedge 1) = 0 \\ \bigvee \\ p_{j,t,m} = (Z_{j,t,m} \wedge Z_{j,t-1,m}) = (1 \wedge 0) = 0 \\ \Rightarrow \\ \sum_{i \in G_j} p_{j,t,i} = p_{j,t,k} + p_{j,t,m} = 0 + 0 = \mathbf{0} \geq \mathbf{1} \\ \textbf{infeasible} \end{array} \right\} \tag{19}$$

The infeasibility can be resolved by activating the constraints violated leading to the situation shown in Figure 6(b). The situation satisfies now the constraints of the new encoding, as shown by expressions (20) and (21) for $t-1$, t and $t+1$.

$$\left\{ \begin{array}{c} p_{j,t,k} = (Z_{j,t,k} \wedge Z_{j,t-1,k}) = (1 \wedge 1) = 1 \\ \bigvee \\ p_{j,t,m} = (Z_{j,t,m} \wedge Z_{j,t-1,m}) = (1 \wedge 0) = 0 \\ \Rightarrow \\ \sum_{i \in G_j} p_{j,t,i} = p_{j,t,k} + p_{j,t,m} = \\ 1 + 0 = 1 \geq 1 \\ \textbf{feasible} \end{array} \right\} \tag{20}$$

$$\left\{ \begin{array}{c} p_{j,t+1,k} = (Z_{j,t+1,k} \wedge Z_{j,t,k}) = (0 \wedge 1) = 0 \\ \bigvee \\ p_{j,t+1,m} = (Z_{j,t+1,m} \wedge Z_{j,t,m}) = (1 \wedge 1) = 1 \\ \Rightarrow \\ \sum_{i \in G_j} p_{j,t+1,i} = p_{j,t+1,k} + p_{j,t+1,m} = \\ 0 + 1 = 1 \geq 1 \\ \textbf{feasible} \end{array} \right\} \tag{21}$$

Figure 7(a) has an optimal path from time steps A to C. However, the improvement introduced here will set a time step B between time steps A and C as shown in Figure 7(b). If we need to reach C directly from a point A and, for any reason, there is no option to include one more time step, our improvement will not return a solution. Thus, the condition to apply the proposed improvement is sufficient, but not necessary for such basic (and constrained) case.



(a) (b)

Fig. 7. Directly path against intermediate path.

C. Non-linear risk allocation

The CNPP encoding described thus far introduces a non-linearity through constraints (7)-(9), which relates risk to the size of the obstacle safety margins. The authors in [14] address this non-linearity by applying the YALMIP branch-and-bound solver to the full disjunctive convex program, and by applying the SNOPT solver to the convex subproblems. To improve efficiency, this paper proposes instead to use a piece-wise linear function to approximate the $\text{erf}^{-1}(x)$. We note that the domain and image of $\text{erf}^{-1}(x)$ are positive for $\Delta \leq 0.5$ and $0 < \delta_{jt} \leq \Delta$ from equation (8) and constraint (12). Further, in the range $1 - 2\Delta \leq x < 1$, as $x \to 1$, the error function $\text{erf}^{-1} \to \infty$. In addition, $\Delta > 0$ for $x = 1 - 2\delta_{jt}$. Furthermore, the function $\text{erf}^{-1}(x)$ increases monotonically, hence it can be bounded by a piece-wise linear curve, specified through a linear program formulation. Figure 8 illustrates an approximation of $\text{erf}^{-1}(x)$ using $N = 8$ linear segments, for $\Delta = 0.1$.

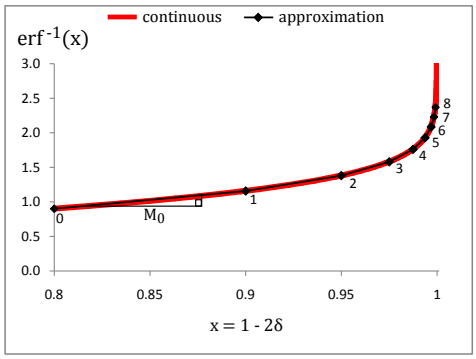


Fig. 8. Piecewise linear approximation for erf^{-1}, where $\delta \leq \Delta = 0.1$ and $N = 8$.

The linear approximation defines values $\bar{y}_n$ in terms of $\bar{x}_n$ through equations (22) and (23), while the slope of each linear segment M_n between points n and $n+1$ is defined by equation (24).

$$\bar{x}_n = 1 - 2\left(\frac{\Delta}{2^n}\right) \quad n = 0, ..., N \tag{22}$$

$$\bar{y}_n = \text{erf}^{-1}(\bar{x}_n) \quad n = 0, ..., N \tag{23}$$

$$M_n = \frac{\bar{y}_{n+1} - \bar{y}_n}{\bar{x}_{n+1} - \bar{x}_n} \quad n = 0, ..., N-1 \tag{24}$$

For $\text{erf}^{-1}(x)$ with $x = 1 - 2\delta_{jt}$, the domain $1 - 2\Delta \leq x < 1$ is divided into N intervals. A variable λ_{njt} is introduced for

each interval $n = 1, ..., N$ and $x = 1 - 2\delta_{jt}$ is defined by Equation (25), where $\bar{x}_0$ specifies the minimum x value. The approximated value for $\text{erf}^{-1}(x)$ is reached by constraints (26) and the bounds for the λ_{njt} variables are given by constraints (27).

$$1 - 2\delta_{jt} = \bar{x}_0 + \sum_{n=0}^{N-1} \lambda_{njt} \qquad \forall j, t \tag{25}$$

$$\text{erf}^{-1}(x) := \bar{y}_0 + \sum_{n=0}^{N-1} M_n \lambda_{njt} \qquad \forall j, t \tag{26}$$

$$0 \leq \lambda_{njt} \leq [\bar{x}_{n+1} - \bar{x}_n] \qquad \forall j, t, n < N \tag{27}$$

$$\lambda_{n+1,jt} > 0 \Rightarrow \lambda_{njt} = [\bar{x}_{n+1} - \bar{x}_n] \qquad \forall j, t, n < N \tag{28}$$

We have $\lambda_{njt} = \bar{x}_{n+1} - \bar{x}_n$ for $n = 0, 1, ..., k-1$; $0 < \lambda_{kjt} < [x_{k+1} - x_k]$ for $n = k$ and $\lambda_{njt} = 0$ for $n = k+1, ..., N-1$. This will always happen once we have a monotonically increasing function inside a minimization problem, so expression (28) holds. Finally, we re-state previous constraints (10) and (12) as follow:

$$c_{i,t}(\delta_{jt}) = \text{erf}^{-1}(1 - 2\delta_{jt})\sqrt{2a_i^T \Sigma_t a_i}$$

$$\Rightarrow c_{i,t}(\delta_{jt}) = (\bar{y}_0 + \sum_{n=0}^{N-1} M_n \lambda_{njt})\gamma_{it} \tag{29}$$

with $\gamma_{it} = \sqrt{2a_i^T \Sigma_t a_i}$.

$$c_{i,t}(\delta_{jt}) + b_i - a_i^T \mu_t \leq \bar{M}(1 - Z_{jti})$$

$$\Rightarrow (\bar{y}_0 + \sum_{n=0}^{N-1} M_n \lambda_{njt})\gamma_{it} + b_i - a_i^T \mu_t \leq \bar{M}(1 - Z_{jti}) \tag{30}$$

$$\sum_j \sum_t \delta_{jt} \leq \Delta \Rightarrow \sum_j \sum_t (1 - \bar{x}_0 - \sum_{n=0}^{N-1} \lambda_{njt}) \leq 2 \cdot \Delta \tag{31}$$

D. Objective functions

In this work, the objective function $g(\cdot)$ is a real-valued function of the controls u_t applied throughout the path. Two candidate objective functions are described here; their performance is evaluated in section VI. The first function is based on an approximation to the norm-2 by a 32 sided polygon as proposed in [14]. Specifically, let's assume a two dimension vector $u_t := [u_t^x, u_t^y]^T$ and a continuous variable r_t. The objective function is then approximated by equation (32), where constraints (33) relate r_t to the approximated values.

$$g(\cdot) = \sum_t r_t \tag{32}$$

$$r_t \geq cos\left(\frac{2\pi n}{32}\right) u_t^x + sin\left(\frac{2\pi n}{32}\right) u_t^y \quad \forall (t, n) \tag{33}$$

The second objective function, which is introduced by this paper, proposes a piece-wise linear approximation for the quadratic objective function described in [5]. For $u_t = [u_t^x \quad u_t^y]^T$, we have $u_t^T \cdot u_t = (u_t^x)^2 + (u_t^y)^2$; this requires computing two quadratic terms, $(u_t^x)^2$ and $(u_t^y)^2$. The objective function is applied to a positive domain, resulting in a monotonically increasing function. This function is approximated by dividing the domain into $N = 32$ equal sub-intervals ($\frac{u_{max}}{N}$), and by defining variables β_{tn}^d for each one of these sub-interval, according to equations (34) to (36). In addition, we introduce variable $h_t^d \geq 0$ with $h_t^d = \left|u_t^d\right|$ for $d \in \{x, y\}$.

$$h_t^d = \left|u_t^d\right| \quad \Leftrightarrow \quad \begin{matrix} h_t^d \geq +u_t^d \\ h_t^d \geq -u_t^d \end{matrix} \qquad \forall (t, d \in \{x, y\}) \tag{34}$$

$$h_t^d = \sum_n \beta_{tn}^d \qquad \forall (t, d \in \{x, y\}) \tag{35}$$

$$0 \leq \beta_{tn}^d \leq \frac{u_{max}}{N} \qquad \forall (t, n, d \in \{x, y\}) \tag{36}$$

The use of modulus is justified in this case since we have a quadratic function for each variable, where it will be enough to approximate only the positive side of its domain. The same idea can be extend to general quadratic objectives by defining $h_t^d = \left|a_t^d u_t^d - b_t^d\right|$ where a_t^d and b_t^d are constants. These constants define, respectively, the slope and the position of the quadratic function. In this case, the function needs to be convex with $a_t^d > 0$ for minimization or $a_t^d < 0$ for maximization problems. Our second objective function is summarized by equation (37) with parameter M_{tn}^d being the angular coefficient for the quadratic function, similar to one presented in section IV-C. The constraints (38) have to be satisfied and they always occur once each term in the objective function (37) is monotonically increasing.

$$g(\cdot) = \sum_t \sum_{d \in \{x,y\}} \sum_n M_{tn}^d \beta_{tn}^d \tag{37}$$

$$\beta_{t,n+1}^d > 0 \Rightarrow \beta_{tn}^d = \frac{u_{max}}{N} \quad \forall (t, n < N, d \in \{x, y\}) \tag{38}$$

E. CNPP encoding

We conclude our encoding discussion by summarizing the two MILP models proposed for CNNP. During the remainder of this paper we use $CNPP^{LE}$ to refer to our Linear Encoding for the original non-linear formulation, proposed by Blackmore et al [14], and $CNPP^{LEN}$ to refer to this Linear Encoding added by the new constraints from section IV-B. The objective function $g(\cdot)$ is quadratic in the CNPP, but the proposed encodings are still MILP since two linear approximation are applied as described in section IV-D.

$CNPP^{LE}$: Min $g(\cdot)$ s.t.{ (1),(3),(11),(30), (31)}
μ_t, u_t, $\lambda_{njt} \geq 0$, $Z_{jti} \in \{0, 1\}$

$CNPP^{LEN}$: Min $g(\cdot)$ s.t.{ (1),(3),(14)-(18),(30),(31)}
μ_t, u_t, $\lambda_{njt} \geq 0$, $Z_{jti} \in \{0, 1\}$, $p_{jti} \geq 0$

If only binary variables are taken into account, we have a combinatorial problem with complexity $O(2^{J \cdot T \cdot N})$, using big O notation in terms of N edges, T time horizons and J obstacles, for both encodings. There are some constraints that avoid

the evaluation of all possibilities. In $CNPP^{LE}$, constraints (11) reduce the combinations on variable Z_{jti} and impose the complexity in terms of edges as $O(2^N - 1) = O(2^N)$. Thus, constraints (11) are not sufficient to reduce the complexity for $CNPP^{LE}$ that will be $O(2^J)$, $O(2^T)$ and $O(2^N)$ in terms of obstacle, time step and edges, respectively, for the worst case. In $CNPP^{LEN}$, constraints (14)-(18) replace constraints (11) and impose the complexity in terms of edges as $O(2^{N-1}) = O(2^N)$. $CNPP^{LEN}$ has less combination than $CNPP^{LE}$ ($2^{N-1} \leq 2^N - 1$), but both encodings have the same complexity in terms of big O notation.

V. Methods

The proposed linear encoding $CNPP^{LE}$ and $CNPP^{LEN}$ allow us to solve instances of the CNPP using exact methods as branch & cut algorithm from CPLEX solver. However, it is indeed possible to apply other relaxation approaches over those encoding. First, this is done over the customized solution approach (CSA), described in [14], which computes upper and lower bound risks from the original non-linear encoding of CNPP. Next, a new method is proposed by the present paper, named hybrid incremental solution approach (HISA), that improves an initial solution returned from Dijkstra algorithm solving MILP sub-models.

A. Customized Solution Approach (CSA)

CSA combines fixed risks with risk allocation approaches as described in [14] and [3]. The fixed risk approaches aim to lower and upper bound the linear encoding. The lower bound variant is named fixed risk relaxation (FRR) and replaces constraints (30) by (39).

$$c_{i,t}(\Delta) + b_{ji} - a_{ji}^T \mu_t \leq \bar{M}(1 - Z_{jti}) \quad \forall (j,t,i \in G_j) \quad (39)$$

This is achieved by assigning to δ_{jt} all of the risk, Δ, specified in the global chance constraint, that is, $\delta_{jt} = \Delta$. Thus the non-linear term $c_{i,t}(\delta_{jt})$ is replaced by constant $c_{i,t}(\Delta)$. Of course, the sum over δ_{jt} violates constraint (12), which is removed in the FRR. While the FRR solution may be infeasible, it lower bounds the cost of the solution and identifies when no solution exists. Therefore, CSA uses FRR as a starting point to a correct and locally optimal method, named the risk allocation approach (RAA). Intuitively, RAA locally adjusts the FRR solution to a solution of the original CNPP. Specifically, the solution to the FRR offers a candidate assignment to the binary variables of the CNPP. This assignment corresponds to deciding which side of each obstacle the vehicle traverses. CSA fixes this binary variable assignment within the original CNPP encoding, while leaving the obstacle risk allocation as decision variables δ_{jt}. RAA then solves the resulting non-linear convex program, to find, if one exists, the optimal risk allocation and trajectory that respects this binary variable assignment.

In this paper, the piece-wise linear approximation introduced allows RAA to solve a linear convex program. To formulate the reduced CNPP, RAA returns constraints (30) only for those obstacles where a risk must be allocated. These obstacles are taken from Z_{ijt} values in the final solution of FRR, as defined by constraints (40).

$$Z_{jti} = 1 \Rightarrow (\bar{y}_0 + \sum_{n=0}^{N-1} M_n \lambda_{njt})\gamma_{jit} + b_{ji} - a_{ji}^T \mu_t \leq 0 \quad (40)$$

The process of computing a lower-bound solution using FRR may not result in a feasible solution. In this event CSA employs a second strategy, in which it computes an upper-bound solution, using a method called fixed risk tightening (FRT). This method upper bounds cost by distributing global risk Δ uniformly across all obstacles. Specifically, FRT replaces constraints (30) by (41):

$$c_{i,t}(\frac{\Delta}{T.J}) + b_{ji} - a_{ji}^T \mu_t \leq \bar{M}(1 - Z_{jti}) \quad \forall (j,t,i \in G_j) \quad (41)$$

FRT preserves feasibility since the risk allocated $\delta_{jt} = \frac{\Delta}{T.J}$ does not violate constraint (12). If FRT returns a feasible solution, CSA will improve it by using RAA. Similar to FRR, this is done by fixing the CNPP binary variables to those from FRT solution, while leaving obstacle risk allocation as decision variables δ_{jt} for RAA. The described processes of finding initial relaxed solutions using FRR or FRT, followed by a local improvement step using RAA, are summarized in Algorithm 1. Also the whole CSA procedure combining FRR, FRT and RAA resolution is illustrated by Figure 9.

Algorithm 1: FRR_FRT_RAA(Model,relaxType)

```
if (relaxType == FRR) then
    Model ← Replace constraints (30) by (39) //Solving FRR;
else
    Model ← Replace constraints (30) by (41) //Solving FRT;
Model ← Remove constraints (31) and variables λ_nijt;
Solve(Model);
if isFeasible(Model.status) then
    if (relaxType == FRR) then
        Model ← Remove constraints (39);
    else
        Model ← Remove constraints (41);
    Model ← Add constraints (40) from values of Model.Z_ijt;
    Model ← Add constraints (31) and variables λ_nijt;
    Solve(Model) //Solving RAA;
return Model;
```

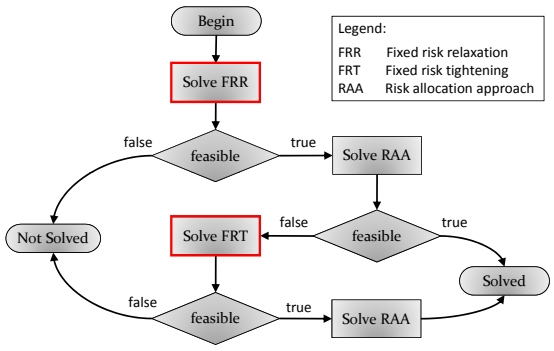


Fig. 9. Overall idea about how CSA combines FRR, FRT and RAA

B. Hybrid incremental solution approach (HISA)

In this section we propose HISA that generates a visibility graph [6], based on relaxed risks, and increases the risk level step by step. The graph is built based on relaxed risk values ($\delta_{jt} = \frac{\Delta}{2^n}$) such that all feasible paths are kept in the visibility graph (G) (see Figure 10), where the dots are nodes ($v \in G$) near vertexes of obstacles.

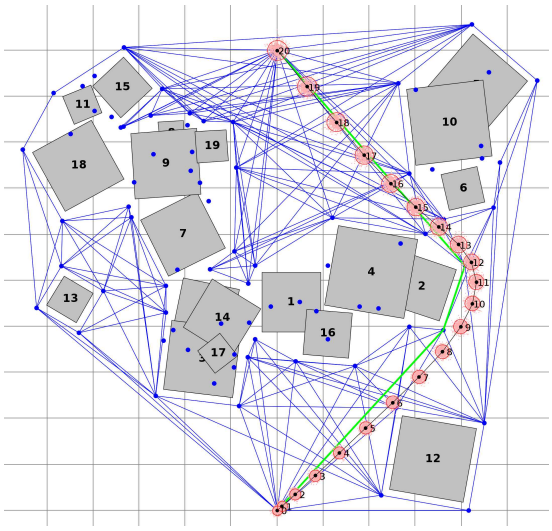


Fig. 10. Graph of the paths and the route of the HISA for $\Delta = 0.001$. The circles give the 99.9% certainty region for the UAV position.

Figure 10 also highlights the reference path, achieved by applying Dijkstra algorithm, and the full path solution, after risk allocation.The reference path allows us to define almost all binary variables from the proposed CNPP encoding, becoming easier to solve the risk allocation problem. HISA does not guarantee solution bounds or even optimal solution as CSA,but returns good solutions within a reduced computational time as reported in the next section. Algorithm 2 describes HISA.

Algorithm 2: HISA

```
1  model ← CNPP^LEN;
2  n ← 0;
3  repeat
4      G ← create-graph(Δ/2^n);
5      path ← Dijkstra(G);
6      submodel ← fix-path(model, path, n);
7      Execute Algorithm1(submodel,FRR) with Δ/2^n;
8      n ← n + 1;
9  until isFeasible(submodel.status) or n ≥ maxIter;
10 if isFeasible(submodel.status) then
11     return model;
12 else
13     return HISA fail;
```

A visibility graph G is first built at line 4, where nodes are defined from obstacle vertexes plus the start ($\hat{x}_0$) and goal ($\hat{x}_{goal}$) nodes. The nodes are positioned from obstacle vertexes by considering relaxed risks and they are obtained solving the equation system (42). This system gives the coordinates of nodes v for obstacle j with hyperplanes $i \in G_j$ and $k \in G_j$ at the nth iteration of HISA algorithm.

$$\begin{aligned} a_i^T \cdot v &= b_i + c_{i,0}(\tfrac{\Delta}{2^n}) \\ a_k^T \cdot v &= b_k + c_{k,0}(\tfrac{\Delta}{2^n}) \end{aligned} \tag{42}$$

All nodes v with collision risk greater than $\frac{\Delta}{2^n}$ are removed because they are infeasible (see nodes without connection in Figure 10). Thus edges are built only among nodes v that satisfy the collision risk $\frac{\Delta}{2^n}$ at each iteration. Next, Dijkstra algorithm is applied at line 5 over G to find the shortest path from the start vertex $\hat{x}_0$ to the goal $\hat{x}_{goal}$. A $submodel$ is created in line 6 by fixing the reference $path$ returned by Dijkstra algorithm. Similar to the path returned by FRR, the reference path allows us to prune the search space by fixing several Z_{ijt} values in the $submodel$. However, other Z_{ijt} variables can be optimized when applying Algorithm 1 with $relaxType = FRR$ at line 7. This reduces the number of binary varibles optimized by FRR and becomes possible to reach a feasible full path from FRR using the reference path provided by Dijkstra algorithm. To reach feasibility for CNPP, the allocated risk made by FRR, $\frac{\Delta}{2^n}$, is reduced at each iteration. Of course, if no feasible solution is found after $maxIter$, HISA will fail. This is the same possibility of fail reported for CSA in [14].

Algorithm 3: fix-path($model$, $path$, n)

```
1  step ← cost(path)/T;
2  S_jt ← ∅                ∀(j,t);
3  for (v,w) ∈ edge(path) do
4      lb ← ⌊v.cost/step⌋;
5      ub ← ⌊w.cost/step⌋;
6      for j ∈ obstacles do
7          for i ∈ G_j do
8              for t ∈ [lb,ub] do
9                  λ ← (t − lb)/(ub − lb);
10                 p ← v ∗ (1 − λ) + w ∗ λ;
11                 if (a_jt^T · p ≥ b_ji + c_i,t(Δ/2^n)) then
12                     S_jt ← S_jt ∪ {i};
13 submodel ← model
14 for t ∈ [0,T] do
15     for j ∈ obstacles do
16         remove from submodel at period t all i ∈ G_j | i ∉ S_jt;
17 return submodel;
```

Algorithm 3 describes the procedure fix-path. The fix-path algorithm builds a $submodel \subseteq model$ to follow a reference $path$ in G. It begins by calculating $step$ as the average distance traveled between two consecutive time steps t in line 1. In lines 2-12, it is defined the set S_{jt} with all hyperplanes $i \in G_j$ that will be in $submodel$, for each obstacle j and time step t. First, the edges $(v \rightarrow w)$ in the $path$ are traveled considering the lower-bound (lb) and upper-bound (ub) time to go through these edges (lines 4-5). Next, for each obstacle j, it is added in S_{jt} the hyperplanes $i \in G_j$ enabled at time steps $t \in [lb, ub]$ from v to w. The parameter Δt of CNPP allows us to define the amount of time steps t available in $[lb, ub]$. Finally, the $submodel$ is built by removing all hyperplanes that are disable in S_{jt} (lines 14-16). It is worth mentioning that CSA and HISA do not guarantee to find a feasible or optimal solution even when there is one. This situation can happen for CSA when

FRT fails, i.e., there is only one possible path but the fixed level of risk imposed by FRT prevents to go through it. HISA applies incremental risks from FRR to FRT instead of a fixed risk. Therefore, HISA tries to distribute risk by incremental steps. Moreover, it is expected that HISA fails less often than CSA since the former works incrementally.

VI. Results

The computational tests in this section run on an Intel i7 computer with 3.4 GHz and 32 GB RAM. The $CNPP^{LEN}$ and $CNPP^{LE}$ encoding as well as heuristics CSA and HISA are developed in Java using the mathematical programming software IBM ILOG CPLEX 12.6 Callable Library. All methods execute within 1 hour of time limit, and for the CSA this total time is divide equally between steps FRR/RAA (limited to 1800 sec) and FRT/RAA (limited to 1800 sec).

The experiments are conducted over 2D and 3D maps. The state vector x_t has the positions p and velocities v for each dimension, while the control vector u_t has accelerations defined by expressions (43) and (44) for 2D and 3D maps, respectively. Expressions (45)-(46) present plant control and covariance matrices for 2D map. The 3D control parameters extends the 2D plant control.

$$x_t := [p_x \ p_y \ v_x \ v_y]^T, \quad u_t := [a_x \ a_y]^T \tag{43}$$

$$x_t := [p_x \ p_y \ p_z \ v_x \ v_y \ v_z]^T, \quad u_t := [a_x \ a_y \ a_z]^T \tag{44}$$

$$A = \begin{pmatrix} 1 & 0 & \Delta t & 0 \\ 0 & 1 & 0 & \Delta t \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, B = \begin{pmatrix} \Delta t^2/2 & 0 \\ 0 & \Delta t^2/2 \\ \Delta t & 0 \\ 0 & \Delta t \end{pmatrix} \tag{45}$$

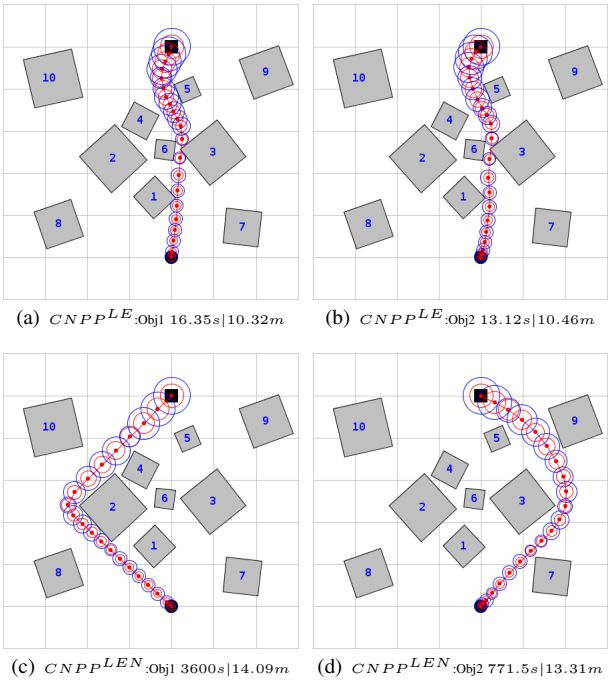
$$\Sigma_{x_0} = \begin{pmatrix} \sigma_{x_0}^2 & 0 & 0 & 0 \\ 0 & \sigma_{x_0}^2 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \Sigma_{w_t} = \begin{pmatrix} \sigma_{w_t}^2 & 0 & 0 & 0 \\ 0 & \sigma_{w_t}^2 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \tag{46}$$

The majority of tests are set with $\sigma_{x_0} = 0.05$ and $\sigma_{w_t} = 0.05 \cdot \sqrt{3/T}$ where T is the number of time steps for a time horizon of 20 seconds and $\Delta t = 20/T$. The controls u_t and velocities v_t are given by $|u_t| \leq u_{max} = 1 \ m/s^2$ and $|v_t| \leq v_{max} = 3 \ m/s$. These parameter values are based on those described in [5], [14]; if any value changes for some experiment, the new value is reported.

A. Objective functions and Encoding Issues

The first experiment illustrates the performance of $CNPP^{LE}$ and $CNPP^{LEN}$ when guided by the two objective functions presented on section IV-D. The 32-sided polygon approximation for the norm-2 is called Obj1 and the piece-wise approximation proposed in this paper for the quadratic function is called Obj2. The tests are executed over a hard map similar to one proposed in [14] with maximum risk $\Delta = 0.001$, time horizon $T = 20$ time steps and number of obstacles $J = 10$. The branch & cut algorithm from solver CPLEX is applied to solve the linear encodings $CNPP^{LE}$ and $CNPP^{LEN}$. Figures11(a)-11(d) show the results achieved.

$CNPP^{LE}$ is faster and returns shorter paths than $CNPP^{LEN}$, once these paths take a shortage through the obstacles 3 and 6 (Figures 11(a) and 11(b)) with risks incurred



(a) $CNPP^{LE}$:Obj1 $16.35s|10.32m$ (b) $CNPP^{LE}$:Obj2 $13.12s|10.46m$

(c) $CNPP^{LEN}$:Obj1 $3600s|14.09m$ (d) $CNPP^{LEN}$:Obj2 $771.5s|13.31m$

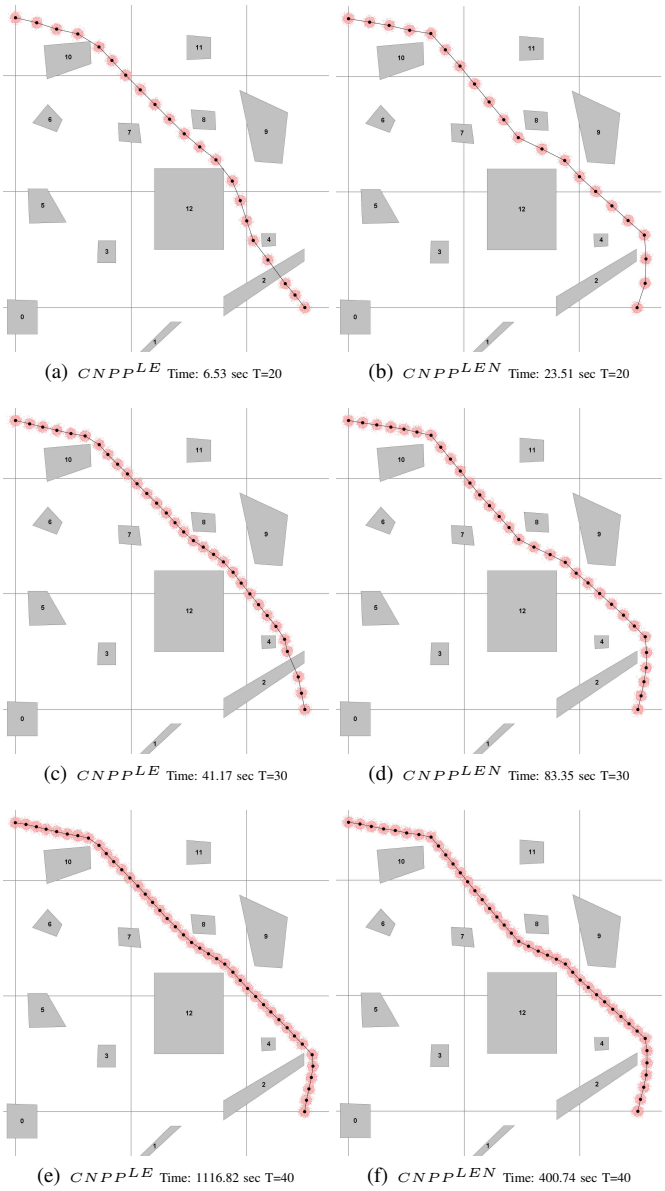
Fig. 11. $CNPP^{LE}$ and $CNPP^{LEN}$ results guided by Obj1 and Obj2.

but not computed. The new constraints in $CNPP^{LEN}$ make its encoding more time consuming with longer paths returned, but no-computed risks are now considered. In this case, paths between obstacles 3 and 6 are avoided, so the risk bound is properly satisfied (Figures 11(c) and 11(d)).

Both models show a better computational time when guided by Obj2. The performance is even better for $CNPP^{LEN}$, where Obj2 reduces considerably the computational time against Obj1. We do not claim that one objective function is always better than other. It is only possible to say that the approximation applied over Obj2, as coded in this papers and evaluated over the hard map illustrated, leads $CNPP^{LEN}$ to be faster than the approximation coded for Obj1. Since the other maps that will be evaluated are generated based on this hard map, the next experiments will be conducted using only Obj2. Also the piece-wise approximation employed in Obj2 can be extended easier to 3D maps than the 32-side polygon for Obj1.

The next experiment emphasizes how the new constraints in $CNPP^{LEN}$ can avoid to overcome obstacles or incur in no-computed risks. We consider a mobile robot that has to avoid obstacles when going from one origin to a destination point. The parameter values and the 2D environment are taken from Rover robot simulations, where the plant model consider $x_t := [p_x \ p_y]^T$ and $u_t := [v_x \ v_y]^T$, it is set $\Delta = 0.01$, $T = 20, 30, 40$, $\Delta t = 20$, $\sigma_{x_0} = 0.1$, $\sigma_{w_t}^x = 0.3$, $\sigma_{w_t}^y = 0.03$ and $v_{max} = 0.5$ m/s. Figures 12(a)-12(f) show the optimal paths returned by $CNPP^{LE}$ and $CNPP^{LEN}$ along with the execution time, when the number of time steps increases.

$CNPP^{LE}$ overcomes obstacles with $T = 20$ and 30 time steps, so the paths are infeasible even though $CNPP^{LE}$ returns them as optimal solutions. The only way to make this encoding to return a path planning, without overcome



(a) $CNPP^{LE}$ Time: 6.53 sec T=20 (b) $CNPP^{LEN}$ Time: 23.51 sec T=20

(c) $CNPP^{LE}$ Time: 41.17 sec T=30 (d) $CNPP^{LEN}$ Time: 83.35 sec T=30

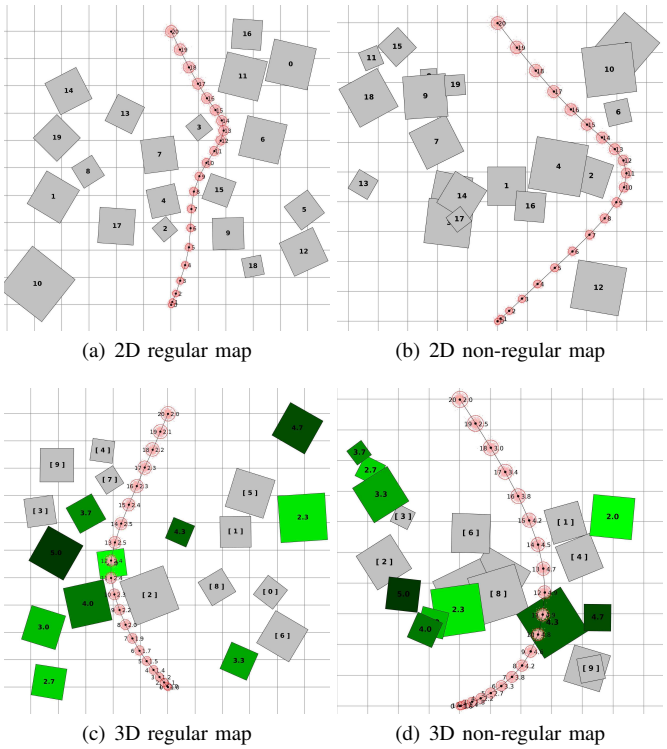
(e) $CNPP^{LE}$ Time: 1116.82 sec T=40 (f) $CNPP^{LEN}$ Time: 400.74 sec T=40

Fig. 12. Solutions and execution time for Rover tests T=20,30,40

obstacles in this scenario, is setting $T = 40$. On the other hand, $CNPP^{LEN}$ finds optimal solutions that do not overcome obstacles for all time steps. $CNPP^{LEN}$ spends more computational time, but improves risk bounds through the paths when the number of time steps increases. Since paths have to be always returned without overcome obstacles and within the maximum level of risk Δ, $CNPP^{LEN}$ is the only encoding able to do this for different times steps in this example. Also, for the only time horizon in which $CNPP^{LE}$ returns a valid solution, $CNPP^{LEN}$ is faster than it to find the same optimal solution (Figures 12(e) and 12(f)).

B. Scaling number of obstacles and time steps

The next computational test evaluates $CNPP^{LEN}$, CSA and HISA for path planning when the number of obstacles and time steps increase. In this experiment, we are comparing an exact solution, solving the full linear encoding $CNPP^{LEN}$, against heuristic solutions returned by CSA and HISA. Two groups of maps are generated: 2D and 3D maps. For each of these groups, two another groups are defined: regular and non-regular maps. The 2D maps have no heights related with the obstacles, so they are always non-fly zones (NFZ). The 3D maps have NFZ as well as obstacles with heights, so the UAV flies over these obstacles. The so-called non-regular maps have obstacles that can be stacked one over another generating irregular shapes regions. These shapes may represent scenarios where we have, e.g, irregular no-fly zones for 2D maps or mountains with different heights at a same area for 3D maps. The regular maps will spread obstacles through the available area, so it can be harder to define a path with several non-fly zones on the map. Figure 13 shows maps for each group.



(a) 2D regular map (b) 2D non-regular map

(c) 3D regular map (d) 3D non-regular map

Fig. 13. Examples for 2D and 3D regular and non-regular maps

A map generator was developed and two large set were randomly created. In the first set of maps, the map generator increased the number of obstacles by $J = 12, 16, 20, 24, 28, 32, 36, 40$ with $T = 20$ and $\Delta = 0.001$, and 50 maps were created for each J value. Thus, a total of 1,600 maps will be evaluated for 2D and 3D regular and non-regular maps. In the second set of maps, 50 maps were generated increasing the time horizon (number of time steps) by $T = 10, 15, 20, 25, 30, 35, 40, 45$ with $J = 20$ and $\Delta = 0.001$. A total of 1,600 maps is also defined for 2D and 3D regular and non-regular maps. Thus, there is a total of 3,000 different maps because we have an overlap for $J = 20$ and $T = 20$. The methods will be evaluated from these maps by three criteria: feasible solutions, computational time and path size. The satisfactory results on all of these criteria are relevant for real-world applications. For instance, when executing a mission, the UAV can demand a new route

(feasible solution) within a short time (computational time). Also, the path size is usually relevant since long paths increase power or fuel consumption.

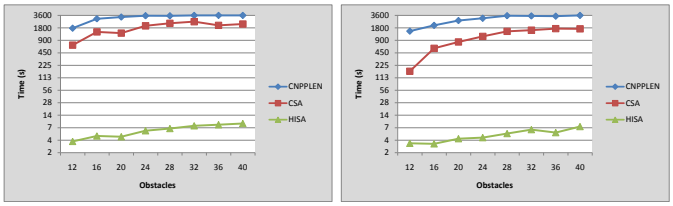
1) Results for 2D maps: Table I has the number of feasible solutions returned by $CNPP^{LEN}$, CSA and HISA for 2D regular and non-regular maps when increasing J and T. The *same maps* row has the amount of maps where all methods find solutions. HISA returns solutions for maps in all groups, except by one regular and another non-regular map with $T = 10$. CSA shows competitive results, while $CNPP^{LEN}$ has problems to return solutions for J and T increasing. The number of obstacles and time steps have a reasonable impact over the amount of decision variables in $CNPP^{LEN}$ encoding, which becomes harder to apply an enumeration algorithm over it.

TABLE I
NUMBER OF FEASIBLE SOLUTIONS FOR 2D MAPS INCREASING J AND T

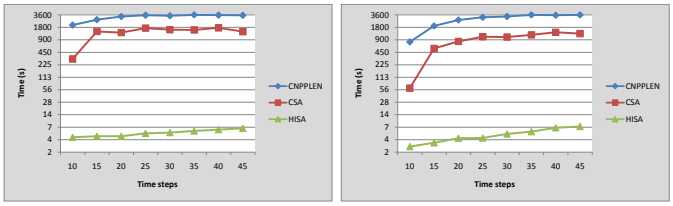
Obstacles (J)	12	16	20	24	28	32	36	40
Regular								
$CNPP^{LEN}$	48	50	48	45	47	34	29	18
CSA	50	50	50	50	50	48	48	47
$HISA$	50	50	50	50	50	50	50	50
same maps	48	50	48	45	47	33	27	17
Non-regular								
$CNPP^{LEN}$	49	50	50	47	47	44	44	36
CSA	50	50	50	50	50	50	50	50
$HISA$	50	50	50	50	50	50	50	50
same maps	49	50	50	47	47	44	44	36
Time Steps (T)	10	15	20	25	30	35	40	45
Regular								
$CNPP^{LEN}$	49	48	48	47	45	37	33	29
CSA	49	49	50	50	50	49	50	47
$HISA$	49	50	50	50	50	50	50	50
same maps	47	47	48	47	45	37	33	28
Non-regular								
$CNPP^{LEN}$	49	50	50	47	45	45	44	44
CSA	50	50	50	50	50	50	50	50
$HISA$	49	50	50	50	50	50	50	50
same maps	48	50	50	47	45	45	44	44

Figures 14(a)-14(d) present the average computational time to return solutions, but taking into account only *same maps* to allow a fair comparison. HISA is the fastest approach spending on average less than 9 sec to define a path for regular and 7.5 sec for non-regular maps (Figures 14(a) and 14(b)). On the other hand, $CNPP^{LEN}$ takes more than 1,700 sec for regular and 1,500 sec for non-regular maps. CSA has a performance better than $CNPP^{LEN}$, but worse than HISA. In Figures 14(c) and 14(d), $CNPP^{LEN}$ also takes longer to return a solution when the number of time steps increases. HISA is again the fastest method, spending less than 6.6 sec on average for all regular maps and less than 7.3 sec for all non-regular maps.

Figures 15(a)-15(d) show average path size values, based on the final trajectories returned for *same maps*. The UAV always starts from (0,0) and goes to (0,-10), thus, the path sizes have values between 10 and 20 units. The path sizes are larger for regular than non-regular maps when increasing J, since their obstacles are more spread through the map. If the number of time steps increases, it is possible to find better path sizes for regular maps. This makes sense since more time steps can lead to a better path planning. HISA returned the shortest paths



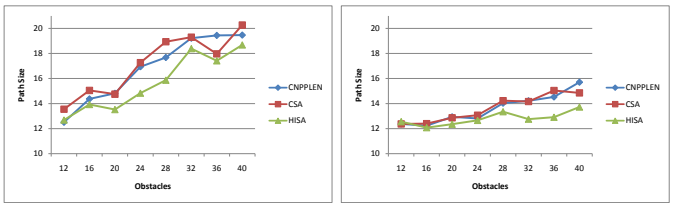
(a) Obstacles for 2D regular (b) Obstacles for 2D non-regular



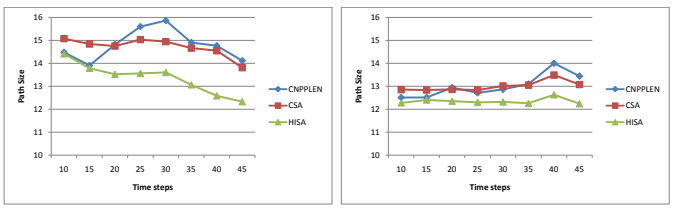
(c) Time steps for 2D regular (d) Time steps for 2D non-regular

Fig. 14. Average computational time when increasing J and T for 2D maps

on average for most of the results, but all approaches reach similar path sizes.



(a) Obstacles for 2D regular (b) Obstacles for 2D non-regular



(c) Time steps for 2D regular (d) Time steps for 2D non-regular

Fig. 15. Average path size when increasing J and T for 2D maps

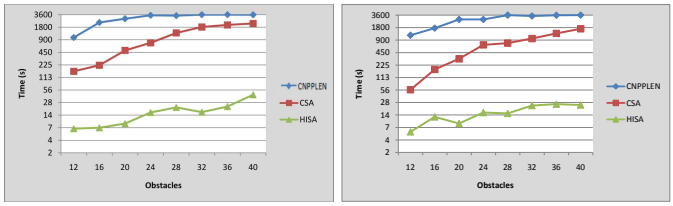
To summarize, CSA is able to return feasible solutions when the number of obstacles and time steps increase, but its drawback is the computational time. $CNPP^{LEN}$ has problems to find feasible solutions when J or T increase within 1h of time limit. HISA seems to be the best approach since it solves the majority of maps, it is computationally fast and reaches competitive path size values.

2) Results for 3D maps: Table II shows the number of feasible solutions returned for regular and non-regular 3D maps increasing J and T. The number of *same maps* is also presented. HISA and CSA return feasible solutions for all groups, except CSA for one non-regular map with $J = 36$. $CNPP^{LEN}$ encoding does not find 19 solutions for both regular and non-regular maps increasing J, and it fails for 11 regular and 12 non-regular maps increasing T. Figures 16(a)-16(d) show the average computational time for *same maps*. HISA is again the fastest method for regular and non-regular maps, while $CNPP^{LEN}$ takes more than 1,800 sec for the

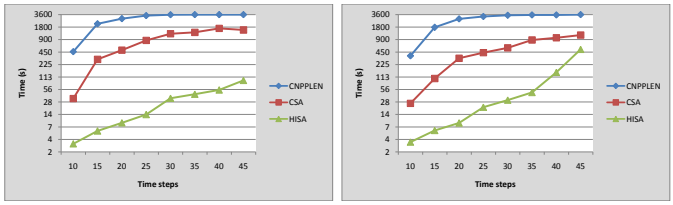
majority of regular and non-regular maps. CSA perfoms better than $CNPP^{LEN}$, but significantly worse than HISA.

TABLE II
NUMBER OF FEASIBLE SOLUTIONS FOR 3D MAPS INCREASING J AND T

Obstacles (J)	12	16	20	24	28	32	36	40
Regular								
$CNPP^{LEN}$	49	49	48	50	45	45	50	45
CSA	50	50	50	50	50	50	50	50
$HISA$	50	50	50	50	50	50	50	50
same maps	49	49	48	50	45	45	50	45
Non-regular								
$CNPP^{LEN}$	50	50	48	47	46	46	50	44
CSA	50	50	50	50	50	50	49	50
$HISA$	50	50	50	50	50	50	50	50
same maps	50	50	48	47	46	46	49	44
Time Steps (T)	10	15	20	25	30	35	40	45
Regular								
$CNPP^{LEN}$	49	47	48	48	49	49	49	50
CSA	50	50	50	50	50	50	50	50
$HISA$	50	50	50	50	50	50	50	50
same maps	49	47	48	48	49	49	49	50
Non-regular								
$CNPP^{LEN}$	50	48	48	50	50	50	46	46
CSA	50	50	50	50	50	50	50	50
$HISA$	50	50	50	50	50	50	50	50
same maps	50	48	48	50	50	50	46	46



(a) Obstacles for 3D regular (b) Obstacles for 3D non-regular

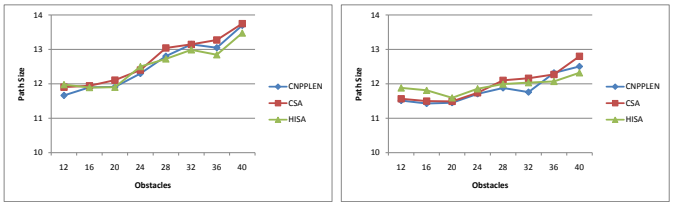


(c) Time steps for 3D regular (d) Time steps for 3D non-regular

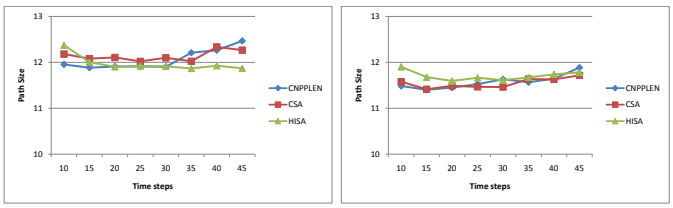
Fig. 16. Average computational time increasing J in 3D maps

HISA seems to have more problems solving 3D maps than 2D maps, where its average execution time is less than 8 sec. This behavior is expected once HISA emphasizes optimization over NFZ, while obstacles with heights are dealt with by solving the related MILP submodel. CSA spends on average half of the time spent by $CNPP^{LEN}$, which is an expected behaviour from a heuristic based on relaxations from $CNPP^{LEN}$ encoding. Figures 17(a) to 17(d) show the path size values, where there is no great difference among these methods. In this case, HISA returns on average path sizes closer to CSA and $CNPP^{LEN}$ when increasing J and T.

The results for 3D maps show that all methods are able to find more feasible solutions, which can be explained by the possibility to fly over obstacles. HISA is again the best approach, returning more solutions within small computational time and with similar path size compared to the others. CSA



(a) Obstacles for 3D regular (b) Obstacles for 3D non-regular

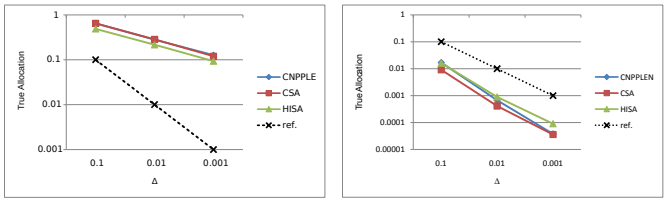


(c) Time steps for 3D regular (d) Time steps for 3D non-regular

Fig. 17. Average computational path size increasing J and T for 3D maps

is competitive against HISA in terms of feasible solutions, but it is also time-consuming solving 3D maps.

3) Monte Carlo simulation: Monte Carlo simulation is applied to evaluate the level of risk incurred by each method when running with and without the new constraints. The experiment is executed over 50 maps with $J = 20$ and $T = 20$, which represent intermediate scenarios. The risk bounds are $\Delta = 0.001, 0.01, 0.1$ and a total of 100,000 paths is generated by Monte Carlo simulation for each method. Figures 18(a) and 18(b) compare the average value of the risk truly incurred. Figure 18(a) shows that $CNPP^{LE}$ as well as HISA and CSA without new constraints fail with incurred risks greater than the maximum risk allowed Δ (ref. in Figures 18(a) and 18(b)). On the other hand, Figure 18(b) has the results running $CNPP^{LEN}$, HISA and CSA with the new constraints. The risk is conservative enough when compared to Δ. In this case, the risk truly incurred is about 10 times more conservative than Δ.



(a) without ($CNPP^{LE}$) (b) with ($CNPP^{LEN}$)

Fig. 18. Monte Carlo validation on all maps with $J = 20$ and $T = 20$

Two main points reported in [14] must be mentioned about the level of conservatism. First, Constraints (10)-(12) provide an approximation for the chance-constrained problem. The approximation is conservative enough to allow solving such problem by applying enumeration techniques to find a global optimal solution. Second, the authors in [14] reported the noticeable conservatism at the corners of obstacles. They explain this conservatism by the similar amount of risk allocated simultaneously to several linear constraints at the corners. This issue is also deeply discussed in [3]. Thus, the

level of conservatism brought by the new encoding can be explained, since our encoding works at the corners to avoid underestimating risks as detailed in Section IV-B.

4) FlightGear simulation: Simulations are executed to illustrate the behaviour of the aircraft following a path returned by HISA. We choose FlightGear (FG) simulator, since it is an open flight simulator with many resources available. The UAV is simulated using the flight dynamic of Cessna 172, but the simulation is not conducted by a human pilot at any moment. An autopilot is coded to control automatically the aircraft flight. Figure 19 shows the main screen of the framework developed to integrates FG, autopilot and execution of HISA. HISA sends a route to autopilot that executes the path from one region to another.

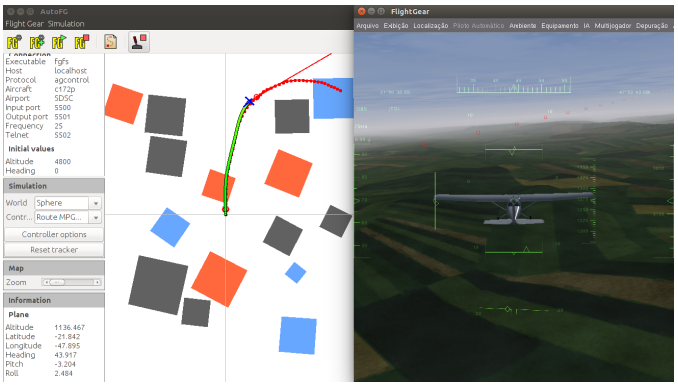


Fig. 19. Integration Flightgear, HISA and autopilot.

We previously define a map that includes an airport location from which the aircraft should take off and land. There are four regions (A, B, C and D), where the aircraft must fly through and six NFZs (gray polygons) as shown by Figure 20. The aircraft must go from the airport to region A and fly next from $A \rightarrow B \rightarrow C \rightarrow D$ when, finally, it goes back to the airport. It is performed a total of 100 simulations taking into account default wind settings of FG (random wind velocities). The uncertainties are set as $\sigma_{x_0} = 30m$ and $\sigma_{w_t} = 10m$, based on previous test with Cessna 172, and total risk $\Delta = 0.001$. All 100 paths followed by the aircraft are shown in Figure 20, where it is possible to see that obstacles avoidance is satisfied within the allowed risk. A shortcut is possible from $B \rightarrow C$, but this would demand irregular maneuvers since there is an inertial velocity to be considered for each destination. The vehicle must also turn around to arrive in the airport heading north. Therefore, we did a small adaptation in the weights of the edges for the Dijkstra algorithm. The weights are defined now by the original Euclidean distance plus the distance traveled to turn and follow the straight line. These weights values mean more than a simple Euclidean distance, and the vehicle will avoid executing irregular maneuvers from one destination to another.

The safety margin from obstacles can be better evaluated from Figure 21. HISA defines a path with minimum expected distance from obstacle of 250.1 meters (red straight line). Figure 21 shows the minimum distance achieved by the aircraft from an obstacle during each simulation. In the worst case, the aircraft becomes 188.9 meters close to the obstacle during simulation 91st. These results indicate that HISA's paths can be conservative enough to avoid collision, keeping reasonable safety margins even under external or internal disturbances. There is a video available at web[1] with one of the simulations executed.

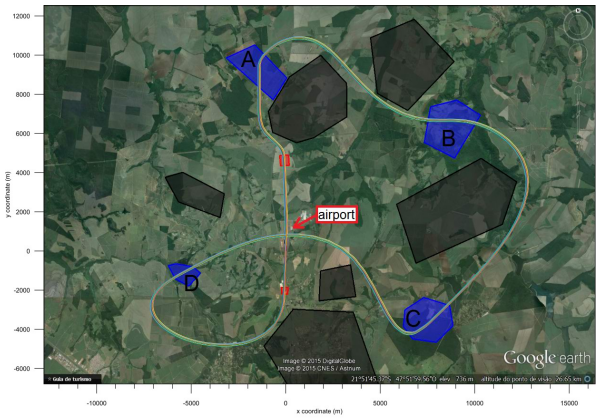


Fig. 20. FlightGear simulator with 100 routes from HISA.

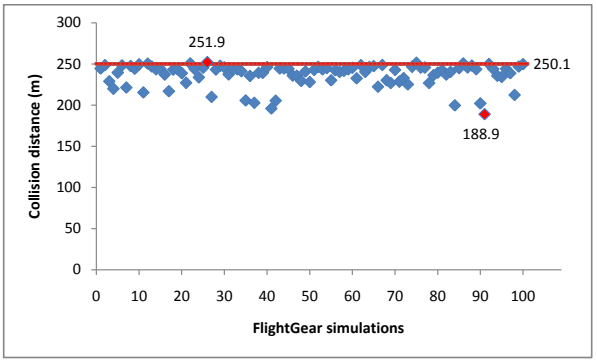


Fig. 21. Minimum collision distance considering all obstacles.

VII. Conclusion

This paper proposes a mixed integer linear programing (MILP) encoding, named as $CNPP^{LEN}$, for the Chance-constrained Non convex Path-planning Problem (CNPP). In this problem, the unmanned aerial vehicle (UAV) path planning problem is addressed for non-convex regions with bounded risk. The proposed encoding deals with the problem of excessive risk during the path planning, which can lead to paths where an obstacle is overcome or many risks incurred are not computed. $CNPP^{LEN}$ addresses these issues without the need of adjust parameters, for example, increasing conveniently the number of time steps within the time horizon as it was done for $CNPP^{LE}$. The new constraints introduced in the $CNPP^{LE}$ encoding solve this problem, once became infeasible to set successive time steps, whose line between them intersect an obstacle. The new encoding introduced on this paper includes constraints to handle obstacle avoidance and risk bounds, along with piece-wise linear approximations for a quadratic objective function and for the inverse of

[1]https://youtu.be/i8flhA4yf90

error function. Moreover, a relevant contribution achieved is to reduce the CNPP from a mixed integer programming (MIP) problem to a mixed integer linear programming problem (MILP). The MILP encoding developed allows us to advance in terms of how to deal with bounded risks as well as parameter scalability. In terms of risks, the paths can not go through obstacles and a better approximation is reached for the total risk incurred following the paths. In terms of scalability, a new benchmark for number of obstacles and time steps is established from a set of 3,000 maps generated for 2D and 3D scenarios, with solutions reported for scenarios including 12 up to 40 obstacles, and 10 up to 45 time steps. These maps can be used by other researchers to validate their methods in problems related to CNPP. The exact solution of the $CNPP^{LEN}$ encoding and the heuristic CSA present different computational time performance. The exact solution of the full encoding is time-consuming, taking usually more than 1800 sec and it does not return solutions for several maps within the time limit. CSA is able to find solutions for the majority of maps under the new encoding, but it is also time-consuming. The proposed HISA heuristic finds solution within a reasonable computational time. Thus, the computational results for several maps as well as the simulation example provided indicate that HISA can work properly when a path planning is needed in a short time. Improvement over the linear encoding and the HISA approach are being developed as a future work, aiming to reach a method faster enough to be embedded in a UAV system for real-time path planning subject to the constraints defined by CNPP. Finally, the next improvements in the proposed encoding and related methods will be compared against other literature methods that use also a safety margin.

ACKNOWLEDGMENTS

This research was supported by Fundacao de Amparo a Pesquisa do Estado de Sao Paulo (FAPESP), projects 2014/12297-0, 2014/11331-0 and 2013/07375-0, MISTI-Brazil Seed Funds with support from the Itau Fund for research on sustainability in Latin America, and the Office of Naval Research Grant N00014-15-IP-00052. The research described in this paper was carried out in part at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration. Research developed using the computational resources of Centro de Ciencias Matematicas Aplicadas a Industria (CeMEAI) financed by FAPESP 2013/07375-0. Finally, the research was also supported by SENAI-Innovation Institute of Embedded Systems.

REFERENCES

[1] A. Richards and J. How, "Aircraft trajectory planning with collision avoidance using mixed integer linear programming," *Proceedings of American Control Conference*, 2002.

[2] E. Fernández-González, E. Karpas, and B. C. Williams, "Mixed discrete-continuous heuristic generative planning based on flow tubes," in *Proceedings of the 24th International Conference on Artificial Intelligence*, ser. IJCAI'15. AAAI Press, 2015, pp. 1565–1572.

[3] M. Ono, "Robust, goal-directed plan execution with bounded risk," Ph.D. dissertation, Massachusetts Institute of Technology, 2012.

[4] A. Richards and J. P. How, "Implementation of robust decentralized model predictive control," *AIAA Guidance, Navigation, and Control Conference and Exhibit*, 2005.

[5] M. Ono, B. C. Williams, and L. Blackmore, "Probabilistic planning for continuous dynamic systems under bounded risk," *J. Artif. Int. Res.*, vol. 46, no. 1, pp. 511–577, Jan. 2013.

[6] Y. Kuwata, "Real-time trajectory design for unmanned aerial vehicles using receding horizon control," Master's thesis, Massachusetts Institute of Technology, 2003.

[7] M. Ono and B. C. Williams, "An efficient motion planning algorithm for stochastic dynamic systems with constraints on probability of failure," in *In Proceedings of the Twentieth National Conference on Artificial Intelligence (AAAI*, 2008.

[8] T. T. Mac, C. Copot, D. T. Tran, and R. D. Keyser, "Heuristic approaches in robot path planning: A survey," *Robotics and Autonomous Systems*, vol. 86, pp. 13 – 28, 2016.

[9] B. Paden, M. Čáp, S. Z. Yong, D. Yershov, and E. Frazzoli, "A survey of motion planning and control techniques for self-driving urban vehicles," *IEEE Transactions on Intelligent Vehicles*, vol. 1, no. 1, pp. 33–55, 2016.

[10] D. Gonzlez, J. Prez, V. Milans, and F. Nashashibi, "A review of motion planning techniques for automated vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 4, pp. 1135–1145, April 2016.

[11] R. Alterovitz, T. T. Simeon, and K. Goldberg, "The stochastic motion roadmap: A sampling framework for planning with markov motion uncertainty," *In Proc. Robotics: Science & Systems*, 2007.

[12] B. D. Luders, M. Kothari, and J. P. How, "Chance constrained rrt for probabilistic robustness to environmental uncertainty," *AIAA guidance, navigation, and control conference*, 2010.

[13] G. S. Aoude, B. D. Luders, J. M. Joseph, N. Roy, and J. P. How, "Probabilistically safe motion planning to avoid dynamic obstacles with uncertain motion patterns," *Autonomous Robots*, vol. 35, no. 1, pp. 51–76, 2013.

[14] L. Blackmore, M. Ono, and B. C. Williams, "Chance-constrained optimal path planning with obstacles," *IEEE Transactions on Robotics*, vol. 27, no. 6, pp. 1080–1094, 2011.

[15] G. A. Hollinger, A. A. Pereira, J. Binney, T. Somers, and G. S. Sukhatme, "Learning uncertainty in ocean current predictions for safe and reliable navigation of underwater vehicles," *Journal of Field Robotics*, vol. 33, no. 1, pp. 47–66, 2016.

[16] J. V. D. Berg, P. Abbeel, and K. Goldberg, "Lqg-mp: Optimized path planning for robots with motion uncertainty and imperfect state information," *The International Journal of Robotics Research*, pp. 895–913, 2011.

[17] A. Bry and N. Roy, "Rapidly-exploring random belief trees for motion planning under uncertainty," in *Robotics and Automation (ICRA), 2011 IEEE International Conference on*. IEEE, 2011, pp. 723–730.

[18] J. van den Berg, S. Patil, and R. Alterovitz, "Motion planning under uncertainty using differential dynamic programming in belief space," in *Robotics Research: The 15th International Symposium ISRR*, vol. 100. Springer, 2016, p. 473.

[19] J. Van Den Berg, S. Patil, and R. Alterovitz, "Motion planning under uncertainty using iterative local optimization in belief space," *The International Journal of Robotics Research*, vol. 31, no. 11, pp. 1263–1278, 2012.

[20] W. Sun, J. van den Berg, and R. Alterovitz, "Stochastic extended lqr for optimization-based motion planning under uncertainty," *IEEE Transactions on Automation Science and Engineering*, vol. 13, no. 2, pp. 437–447, 2016.

[21] S. Tang and V. Kumar, "Mixed integer quadratic program trajectory generation for a quadrotor with a cable-suspended payload," *IEEE International Conference on Robotics and Automation*, 2015.

[22] L. Yang, J. Qi, D. Song, J. Xiao, J. Han, and Y. Xia, "Survey of robot 3d path planning algorithms," *Journal of Control Science and Engineering*, vol. 2016, p. 5, 2016.

[23] T. Schouwenaars, "Safe trajectory planning of autonomous vehicles," Ph.D. dissertation, Massachusetts Institute of Technology, 2005.

[24] K. F. Culligan, "Online trajectory planning for uavs using mixed integer linear programming," Ph.D. dissertation, Massachusetts Institute of Technology, 2006.

[25] L. Blackmore and M. Ono, "Convex chance constrained predictive control without sampling," *Proceedings of the AIAA Guidance, Navigation, and Control Conference*, 2009.

[26] E. F. T. Schouwenaars, B. D. Moor and H. J., "Mixed integer programming for multi-vehicle path planning," *Proceedings of the European Control Conference*, 2001.

[27] R. Deits and R. Tedrake, "Efficient mixed-integer planning for uavs in cluttered environments," *IEEE International Conference on Robotics and Automation*, 2015.



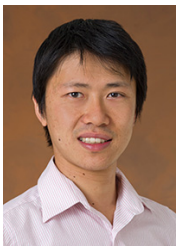
Marcio da Silva Arantes Marcio da Silva Arantes received the BS degree in Computer Science from the Federal University of Lavras in 2012, the S.M. and PhD degree in Computer Science and Computational Mathematics from the University of Sao Paulo (USP) at 2014 and 2017 respectively. Currently, he is working as researcher at SENAI Innovation Institute of Embedded Systems, Florianopolis. He is engaged in applying knowledge gained from mathematical modeling and evolutionary algorithms on industry 4.0 and UAVs applications.



Claudio Fabiano Motta Toledo Claudio Fabiano Motta Toledo received his B.S. in Applied and Computational Mathematics from the State University of Campinas, M.S. and Ph.D. degrees in Electrical Engineering from the State University of Campinas. He was Visiting Professor at the Computer Science and Artificial Intelligence Laboratory of the Massachusetts Institute of Technology (2015). He is a professor at the University of So Paulo since 2011. Toledo has experience in Computer Science with emphasis on Evolutionary Systems and Mathematical Modeling applied to problems in production planning, optimization, unmanned aerial vehicles, digital games, image recovery, among others.



Brian Williams Brian Williams received his S.B., S.M and Ph.D. from MIT in Computer Science and Electrical Engineering in 1989. He pioneered multiple fault, model-based diagnosis in the 80s through the GDE and Sherlock systems at the Xerox Palo Alto Research Center, and model-based autonomy in the 90s through the Livingstone model-based health management and the Burton model-based execution systems. At the NASA Ames Research Center from 1994 to 99 he formed the Autonomous Systems Area, and co-invented the Remote Agent model-based autonomous control system, which received a NASA Space Act Award in 1999. He was a member of the NASA Deep Space One probe flight team, which used remote agent to create the first fully autonomous, self-repairing explorer, demonstrated in flight in 1999.



Masahiro Ono Masahiro Ono is a Research Technologist in the Robotic Mobility Group at Jet Propulsion Laboratory, California Institute of Technology. His research interest includes risk-aware autonomy that enables unmanned probes to reliably operate in highly uncertain environments. He has supported autonomy algorithm development tasks in Mars 2020 Rover and Europa Lander projects. Before joining JPL in 2013, he was an assistant professor at Keio University. He earned Ph.D. and S.M. degrees in Aeronautics and Astronautics as well as an S.M. degree in Technology and Policy from MIT, and a B.S. degree in Aeronautics and Astronautics from the University of Tokyo.